Distributed Graph Clustering

Modularity and Map Equation at Scale

Nitin Gupta

CSC 502 – Systems for Massive Datasets
University of Victoria

Spring, 2026

The Paper

Distributed Graph Clustering using Modularity and Map Equation

Hamann, Strasser, Wagner, Zeitz
Karlsruhe Institute of Technology
Euro-Par 2018

Why I picked this paper:

- Clear big-data problem: graph too large for one machine
- MapReduce-style algorithm with step-by-step data flow
- Two quality measures with clean math
- Strong experimental results on real datasets
- Code + datasets publicly available

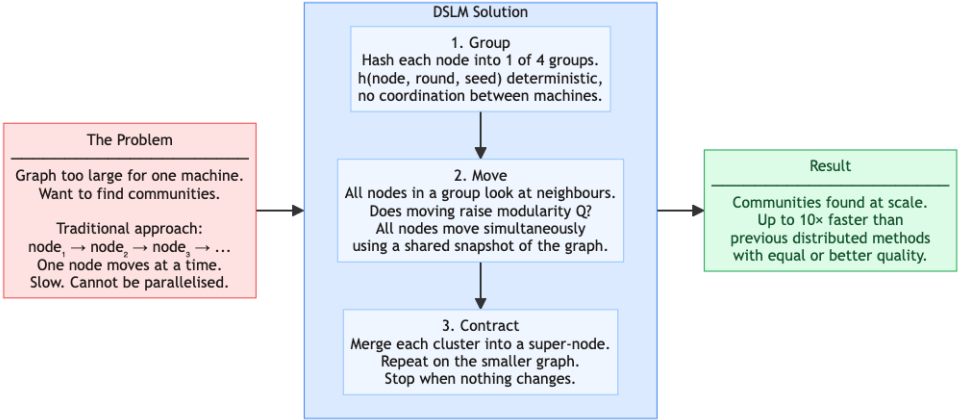
Scale:

- Up to 105M nodes
- Up to 3.3B edges
- 32-host cluster

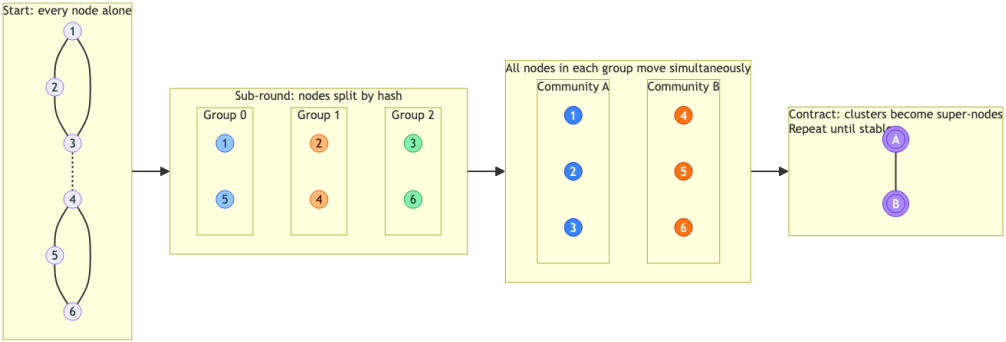
Algorithms:

- DSLM-Mod (modularity)
- DSLM-Map (map eq.)

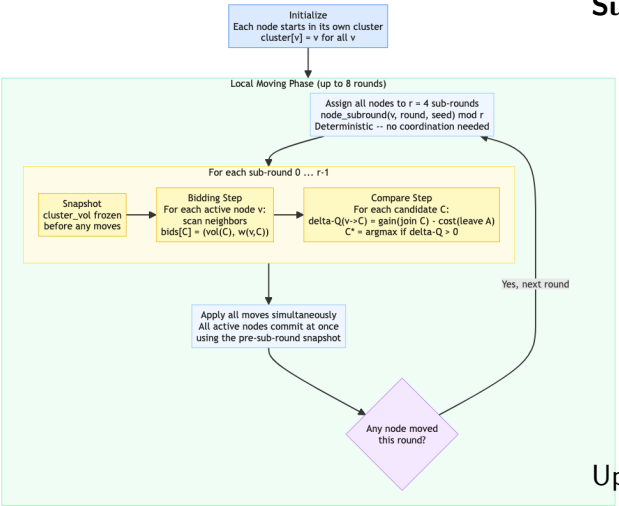
The Problem


The Problem
Graph too large for one machine.
Want to find communities.
Traditional approach:
$node_1 \rightarrow node_2 \rightarrow node_3 \rightarrow \ldots$
One node moves at a time.
Slow. Cannot be parallelised.
DSLM Solution
1. Group
Hash each node into 1 of 4 groups.
h(node, round, seed) deterministic,
no coordination between machines.
2. Move
All nodes in a group look at neighbours.
Does moving raise modularity Q?
All nodes move simultaneously
using a shared snapshot of the graph.
3. Contract
Merge each cluster into a super-node.
Repeat on the smaller graph.
Stop when nothing changes.
Result
Communities found at scale.
Up to 10× faster than
previous distributed methods
with equal or better quality.

What the Algorithm Looks Like



Algorithm Flow: Local Moving Phase

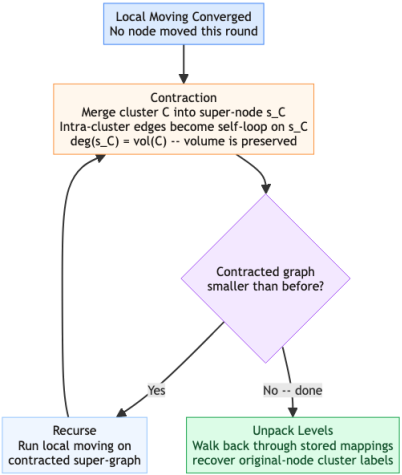


Sub-round parallelization:

- Hash assigns each node to one of 4 sub-rounds deterministically
- Cluster volumes are snapshotted before the sub-round begins
- All active nodes bid and commit simultaneously using the frozen snapshot
- Eliminates the sequential dependency that makes Louvain hard to parallelize

Up to **8 rounds** per level before contraction.

Algorithm Flow: Contraction and Unpacking

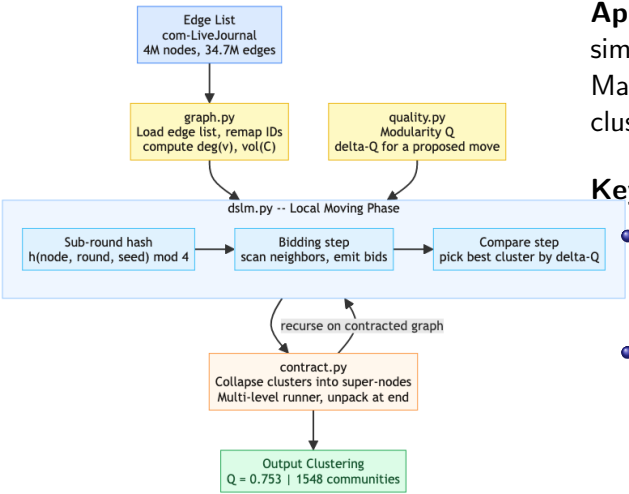


Why contraction matters:

- Each cluster collapses into one super-node
- Graph shrinks dramatically: LiveJournal went from 4M nodes to 280K after level 0
- Self-loops must be preserved so super-node degree equals the original cluster volume
- Recurse until the graph stops shrinking, then unpack all levels

Our run used **6 contraction levels** before converging.

How I Coded It



Approach: Sequential Python simulation that mirrors the distributed MapReduce data flow. No actual cluster needed.

Key optimizations:

- Fast CSV loading with pandas (52 s vs. several minutes for naive row iteration)
- Incremental cluster volumes: update on each move instead of full recompute

Results vs. the Paper

com-LiveJournal (3.99M nodes, 34.7M edges)

Method	Q	Time
Louvain (paper)	0.713	99 s
PLM (paper)	0.713	25 s
DSLM-Mod (paper)	0.710	31 s
DSLM-Mod w/o Cont. (paper)	0.622	14 s
NetworkX Louvain (ours)	0.746	7418 s
Our DSLM-Mod (Python)	**0.753**	1119 s

- $Q = 0.753$ beats all paper baselines; 6 contraction levels give a deeper search than a single pass
- Time gap reflects Python on one laptop vs. compiled C++ on a 32-host cluster, not algorithmic inefficiency

Conclusion

What the paper contributes:

- Sub-round hash breaks sequential dependency in local moving
- First distributed algorithm matching Louvain quality at scale
- Handles 100M+ nodes where GossipMap crashes or times out
- Up to 10× faster than prior distributed methods

What I found during implementation:

- Self-loops during contraction are critical: dropping them collapses all clusters
- 8 rounds per level captures almost all quality (+0.002 for unlimited rounds, 5.8× slower)
- Algorithm is easier to implement correctly than Louvain because the data flow is explicit